

Unterlagen für das obligatorische Fach Informatik in der Klasse 1bW

Jacques Mock Schindler

21. September 2026

Inhaltsverzeichnis

Obligatorisches Fach Informatik (1bW)	4
Programm	4
Hilfsmittel für den Unterricht	4
Beurteilung	5
I. Einführung	6
1. 'Organisation' von Information auf einem Computer	7
1.1. In welcher Form Daten auf dem Computer abgelegt werden	7
1.2. Die Organisation des Massenspeichers unter Windows	8
1.3. Die Dateiadresse	9
2. Einrichten der Arbeitsumgebung	10
2.1. Installation von Python	10
2.2. Installation von Visual Studio Code (VS Code)	11
2.3. Das erste eigene Jupyter Notebook	11
3. Eine sehr kurze Einführung in Python	17
3.1. Rechnen mit Python	17
3.2. Werte Variablen zuweisen	18
3.3. Einfache Python Funktionen aufrufen	20
4. Problemlösung in der Informatik	22
4.1. Ausgangslage	22
4.2. Beispiel: Tricolore	23
4.3. Aufgabe: Österreichische Flagge	26
4.4. Aufgabe: Schweizerfahne	27
4.5. Musterlösungen	28
5. Wiederholungen in Python (For-Schleifen)	31
5.1. Anwendungsübung zu Wiederholungen in Python mit PyTamaro	32
6. Bedingungen in Python	40
6.1. Der nicht -Operator ($\neg$)	41
6.2. Der und -Operator ($\wedge$)	42
6.3. Der oder -Operator ($\vee$)	42
6.4. Anwendungsbeispiel	42

7. Farbstiftschachtel (verschachtelte for-schleifen)	47
7.1. Kartonschachtel 6 wasservermalbare Farbstifte SCHOOL LINE	48
7.2. Etui 120 Farben SUPRACOLOR Aquarelle	48

Obligatorisches Fach Informatik (1bW)

In diesem Repository werden die Unterlagen für das obligatorische Fach Informatik zur Verfügung gestellt.

Programm

Das Programm entspricht dem aktuellen Stand der Planung. Es kann zu Änderungen kommen.

Datum	Thema
18.08.2026	Einstieg und Arbeitsorganisation
25.08.2026	Arbeitsumgebung Jupyter Notebooks
01.09.2026	Problemlösung: Flaggen zeichnen I
08.09.2026	Problemlösung: Flaggen zeichnen II
15.09.2026	Variablen in Python I
22.09.2026	Variablen in Python II
20.10.2026	Wiederholungen: For-Loops
27.10.2026	For-Loops: Anwendungsübung
03.11.2026	Programmverzweigungen
10.11.2026	Einfache Funktionen und Prüfungsvorbereitung
17.11.2026	Prüfung 1: Python-Grundlagen
24.11.2026	Datenstrukturen: Listen I
01.12.2026	Datenstrukturen: Listen II
08.12.2026	Datenstrukturen: Listen III
15.12.2026	Datenstrukturen: Dictionary
05.01.2027	Beurteilung von Algorithmen
12.01.2027	Prüfungsvorbereitung 2
19.01.2027	Prüfung 2: Listen, Dictionaries, Sortieralgorithmen
26.01.2027	Divide and Conquer: Rekursion
02.02.2027	Rekursion II und Semesterabschluss

Hilfsmittel für den Unterricht

Der Unterricht findet über weite Strecken am Computer statt. Die Anleitungen für die Installation von Python bzw. der Jupyter Notebooks sind in der Seitenliste aufrufbar.

Beurteilung

Die Note wird aus dem Durchschnitt der beiden schriftlichen Prüfungen sowie der Benotung der mündlichen Beteiligung berechnet. Der Durchschnitt der Prüfungen zählt zu 90%, die mündliche Beteiligung zu 10%.

Ab dem zweiten Semester wird konstruktive Kritik an den Unterlagen, welche durch ein mir zugewiesenes Issue (der Zugang findet sich unter dem GitHub-Logo  in der Kopfzeile) eingebracht wird, mit maximal einem Notenpunkt auf die jeweils thematisch passende Prüfung angerechnet.

Falls jemand eine persönliche Besprechung wünscht, kann hier einen Termin für eine Sprechstunde reserviert werden (Rent-A-Mock).

Teil I.

Einführung

1. 'Organisation' von Information auf einem Computer

Um eine wissenschaftliche Disziplin zu beschreiben, hilft es, wenn man sich überlegt, welche Frage sie in ihrem Kern zu beantworten versucht. Diese Grundfrage lautet für die Informatik:

Kann diese Aufgabe algorithmisch gelöst werden?

Bevor wir uns dieser Grundfrage nähern, müssen wir allerdings klären, in welcher Umgebung Algorithmen verarbeitet werden. In der Informatik ist das der Computer.

Ein Algorithmus muss Daten lesen, verändern und abspeichern können. Aus Sicht der Software wird Information dabei auf dem Computer primär in zwei grundlegenden Kategorien verwaltet:

- Im Arbeitsspeicher (RAM): Das ist das Kurzzeitgedächtnis des Computers. Hier liegen alle Daten, mit denen gerade im Moment aktiv gerechnet wird. Er ist extrem schnell, aber flüchtig (sobald der Strom weg ist, sind die Daten gelöscht).
- Im Massenspeicher (Festplatte/SSD): Das ist das Langzeitgedächtnis. Hier lagern Betriebssystem, Programme und Dokumente dauerhaft. Er ist langsamer als der Arbeitsspeicher, behält die Daten aber auch, wenn der Computer ausgeschaltet wird.

1.1. In welcher Form Daten auf dem Computer abgelegt werden

Information bzw. Daten werden auf dem Computer als Datei (engl. File) gespeichert. Dabei werden im Alltag zwei grundlegende Arten von Dateien unterschieden:

- Binärdateien
- Textdateien

1.1.1. Binärdateien

Binärdateien sind Dateien, die nur mit einem spezifischen Programm sinnvoll gelesen und bearbeitet werden können. Ein Beispiel für eine Binärdatei ist eine Word-Datei. Die Datei `text.docx` kann nur in Word oder in Programmen, welche Word-kompatibel sind, korrekt angezeigt und bearbeitet werden. Ein anderes Beispiel ist eine PDF-Datei (`text.pdf`), die einen PDF-Reader benötigt. Versucht man, eine solche Datei in einem einfachen Texteditor zu öffnen, sieht man meist nur einen unleserlichen Buchstabensalat.

1.1.2. Textdateien

Eine Textdatei kann im Gegensatz dazu in jedem beliebigen Texteditor geöffnet und sofort gelesen werden. Ein Texteditor ist ein einfaches Programm, mit welchem reiner Text verarbeitet wird. Reiner Text zeichnet sich durch das Fehlen jeglicher optischer Formatierungen (wie Fettgedrucktes, unterschiedliche Schriftgrößen oder Farben) aus. Entsprechend besitzen diese Editoren auch keine Formatierungsmenüs. Optisch erinnern sie ein bisschen an ein weisses Blatt Papier in einer Schreibmaschine.

Genau genommen bestehen alle Dateien aus Bits; bei Textdateien folgt deren Interpretation aber einem allgemein bekannten Standard.

Texteditoren sind in allen Betriebssystemen vorinstalliert:

- Unter Windows heisst dieses Standardprogramm schlicht Editor (bzw. Notepad).
- Auf dem Mac nutzt man dafür TextEdit (wobei man das Programm erst über das Menü „Format“ in den Modus „In reinen Text umwandeln“ versetzen muss).
- Wer auf einem Linux-Rechner arbeitet, nutzt meist Editoren wie Nano, Gedit oder Vim.

1.2. Die Organisation des Massenspeichers unter Windows

Text- und Binärdateien werden im Massenspeicher abgelegt. Die nächste Hürde, die es zu überwinden gilt, ist die Organisation der Dateiablage. Auf einem Computer sind unzählige Dateien gespeichert. Um den Überblick zu behalten, ist der Massenspeicher hierarchisch in einer Ordnerstruktur (auch Verzeichnisstruktur genannt) organisiert.

Unter Windows 11 wird jedes Speichergerät mit einem Laufwerksbuchstaben, gefolgt von einem Doppelpunkt, gekennzeichnet.

1.2.1. Der Start bei „C:“ – Ein historisches Überbleibsel

Wer den Windows-Explorer öffnet, sieht fast immer ein Hauptlaufwerk namens **C:** (oft beschriftet mit „Lokaler Datenträger“). Auf diesem Laufwerk ist das Betriebssystem Windows 11 selbst sowie all deine installierten Programme und persönlichen Daten abgelegt.

Dass das Hauptlaufwerk den Buchstaben **C** trägt, liegt an der technischen Entwicklungsgeschichte der Computerspeicher. Laufwerk **A:** und **B:** waren Anfangs der 1980er-Jahre für Diskettenlaufwerke (*Floppy Disks*) reserviert. Speicher war extrem teuer, Computer verfügten daher noch nicht über interne Festplatten. Das Betriebssystem wurde daher über eine Diskette im Laufwerk **A:** gestartet. Die eigenen Dateien speicherte man auf einer zweiten Diskette im Laufwerk **B:**. Laufwerk **C:** kam erst ins Spiel, als die ersten Festplatten für PCs auf den Markt kamen. Da **A:** und **B:** bereits fest für die Disketten verplant waren, bekam die Festplatte einfach den nächsten freien Buchstaben: das **C:**.

1.2.2. Weitere Buchstaben im Alphabet

Wenn zusätzliche Speichermedien am Computer angeschlossen werden, verteilt Windows die nächsten freien Buchstaben im Alphabet einfach weiter. D:, E:, F:... werden für weitere eingebaute Festplatten, DVD-Laufwerke oder eingesteckte USB-Sticks und externe Festplatten verwendet. Netzlaufwerke, wie beispielsweise ein Schulserver, werden ebenfalls als Laufwerke eingebunden. Um Verwechslungen zu vermeiden, werden dafür oft Buchstaben vom Ende des Alphabets (wie z. B. Z:) verwendet.

1.3. Die Dateiadresse

Damit eine Datei auf dem Computer angezeigt bzw. bearbeitet werden kann, muss sie über eine Adresse bzw. den sogenannten Pfad angesprochen werden können.

1.3.1. Wie ein Pfad aufgebaut ist

Ein typischer Pfad unter Windows 11 sieht beispielsweise so aus:

```
C:\Schule\Informatik\hausaufgabe.txt
```

Von links nach rechts gelesen, beginnt der Pfad mit dem Buchstaben des Laufwerks. Anschließend kommt ein Ordner (hier **Schule**) und dann ein Unterordner (hier **Informatik**) und dann die Datei (hier **hausaufgabe.txt**). Die Anzahl der Unterordner ist dabei theoretisch unbegrenzt. Windows trennt dabei die einzelnen Ebenen durch einen Backslash (\) voneinander ab.

Windows nutzt traditionell den nach links geneigten Backslash (im Schweizerdeutschen Tastaturlayout erzeugt über die Tastenkombination **Alt Gr + <**). Das ist ein historisches Erbe aus den Urzeiten von Microsofts Betriebssystem DOS. Fast der ganze Rest der Computerwelt - darunter das Internet (URLs wie [<https://google.com/suche>] (<https://google.com/suche>); eine URL funktioniert nach demselben Prinzip wie ein Dateipfad: eine hierarchische Adresse), macOS, iPhones und Linux - nutzt den nach rechts geneigten normalen Schrägstrich (/).

2. Einrichten der Arbeitsumgebung

Im Unterricht arbeiten wir mit der Programmiersprache Python. Über weite Strecken verwenden wir sogenannte Jupyter Notebooks, in denen Text und ausführbare Programmteile (Code-Snippets) direkt kombiniert werden können.

Im Hintergrund sind diese Notebooks wie reine Textdateien aufgebaut. Damit wir sie im Unterricht komfortabel und übersichtlich bearbeiten können, nutzen wir einen modernen Editor: Visual Studio Code (kurz VS Code). Diese Software hat sich weltweit als Standard etabliert.

Im Folgenden wird die Installation der erforderlichen Programme Schritt für Schritt erklärt.

2.1. Installation von Python

In der Grundinstallation von Windows 11 ist Python nicht installiert. Hier finden Sie die Schritt für Schritt Anleitung zur Installation von Python in der im Unterricht verwendeten Version.

1. Rufen Sie die Seite `python.org` auf.
2. Wählen Sie den Menüpunkt **Downloads**.
3. Klicken Sie auf die Schaltfläche **Download Python install manager**. In der Grundeinstellung wird der Python-Installer im Ordner **Downloads** abgelegt. Allenfalls öffnet sich ein Dialogfenster, in dem der Speicherort ausgewählt werden kann. Wählen Sie den Ordner **Downloads**.
4. Öffnen Sie den Dateimanager (Windows Explorer) und navigieren Sie in den Ordner **Downloads**.
5. Starten Sie die Installation durch Doppelklick auf die Datei `python-manager-xx.x`. Im sich öffnenden Dialogfenster klicken Sie auf die Schaltfläche **Install Python**.
6. Der Installer stellt im Terminal mehrere Fragen. Beantworten Sie alle mit **y** und **Enter**. Einzige Ausnahme: Die Frage, ob die Online-Hilfe angezeigt werden soll, beantworten Sie mit **n**.
7. Kontrollieren Sie die Installation. Öffnen Sie dazu ein Terminal (Windows-Menü, nach **Terminal** suchen, **Öffnen** klicken). Im sich öffnenden Terminal geben Sie `python` ein und bestätigen die Eingabe durch die Enter-Taste. Im Terminal erscheinen auf der untersten Zeile `>>>` als Eingabeaufforderung. Schreiben Sie

```
print('Hello World')
```

bestätigen Sie diese Eingabe durch die Enter-Taste. Im Terminal sollte `Hello World` ausgegeben werden. Wenn das geklappt hat, haben Sie erfolgreich Ihren ersten Python-Befehl ausgeführt. Sie können `Python` beenden, indem Sie `quit()` eingeben und durch die Enter-Taste bestätigen. Das Terminal schliessen Sie durch die Eingabe von `exit` und die Bestätigung durch die Enter-Taste.

2.2. Installation von Visual Studio Code (VS Code)

1. Rufen Sie die Seite <https://code.visualstudio.com/download> (kurz-URL <https://t1p.de/jt3ge>) auf.
2. Klicken Sie die Schaltfläche **Windows**. In der Grundeinstellung wird der VS Code-Installer im Ordner **Downloads** abgelegt. Allenfalls öffnet sich ein Dialogfenster, in dem der Speicherort ausgewählt werden kann. Wählen Sie den Ordner **Downloads**.
3. Öffnen Sie den Dateimanager (Windows Explorer) und navigieren Sie in den Ordner **Downloads**.
4. Starten Sie die Installation durch Doppelklick auf die Datei **VSCodeUserSetup-x64-xxx**. Im sich öffnenden Dialogfenster akzeptieren Sie die Lizenzbedingungen und klicken Sie auf **Weiter**.
5. Akzeptieren Sie den Standard-Ziel-Ordner durch **Weiter**.
6. Akzeptieren Sie den Startmenü-Ordner mit **Weiter**.
7. Im Dialog **Zusätzliche Aufgaben auswählen** kreuzen Sie die vier untersten Boxen an.
8. Bestätigen Sie alle getroffenen Wahlmöglichkeiten durch **Installieren**.
9. Schliessen Sie die Installation durch klicken auf **Fertigstellen** ab.
10. Wenn sich VS Code zum ersten Mal öffnet, klicken Sie **Continue without Signing in**.
11. Wählen Sie im nächsten Dialog ein Farbschema, das Ihnen gefällt und bestätigen Sie Ihre Wahl durch einen Klick auf **Continue**.
12. Schliessen Sie das Set-Up ab durch anklicken von **Get Started**.

2.3. Das erste eigene Jupyter Notebook

In der Einleitung habe ich einen grundlegenden Überblick über die Dateiorganisation gegeben. Die dort beschriebene Struktur können Sie beim Erstellen des ersten eigenen Jupyter Notebooks nutzen. Das stellt sicher, dass Sie die einmal erstellte Datei auch wieder finden.

Ich gehe davon aus, dass Sie die folgende Ordnerhierarchie verwenden:

```
C:\Users\username\Documents\Schule\Informatik
```

`username` ist durch Ihren Windows-Benutzername zu ersetzen. Falls Sie die entsprechenden Ordner noch nicht angelegt haben, ist jetzt der Zeitpunkt, das zu tun. Legen Sie im Ordner

Informatik zusätzlich einen Unterordner mit dem Namen **YYMMDD_einfuehrung** an; für den 19. August 2026 also **260819_einfuehrung**.

Wenn der Ordner angelegt ist, öffnen Sie VS Code (Windows Menü > VS Code).

Das VS Code Fenster ist im ersten Moment etwas verwirrend.

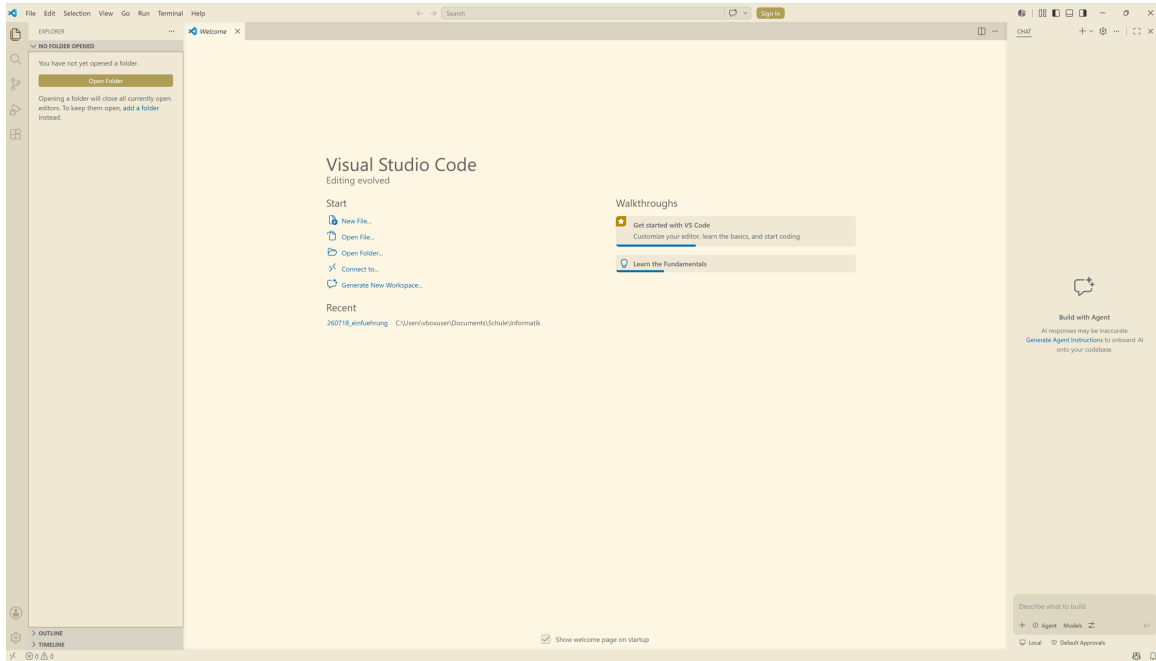


Abbildung 2.1.: VS Code Fenster beim Start.

Gehen wir die einzelnen Elemente des Fensters der Reihe nach durch. Im Zentrum des VS Code-Fensters findet sich der Arbeitsbereich.

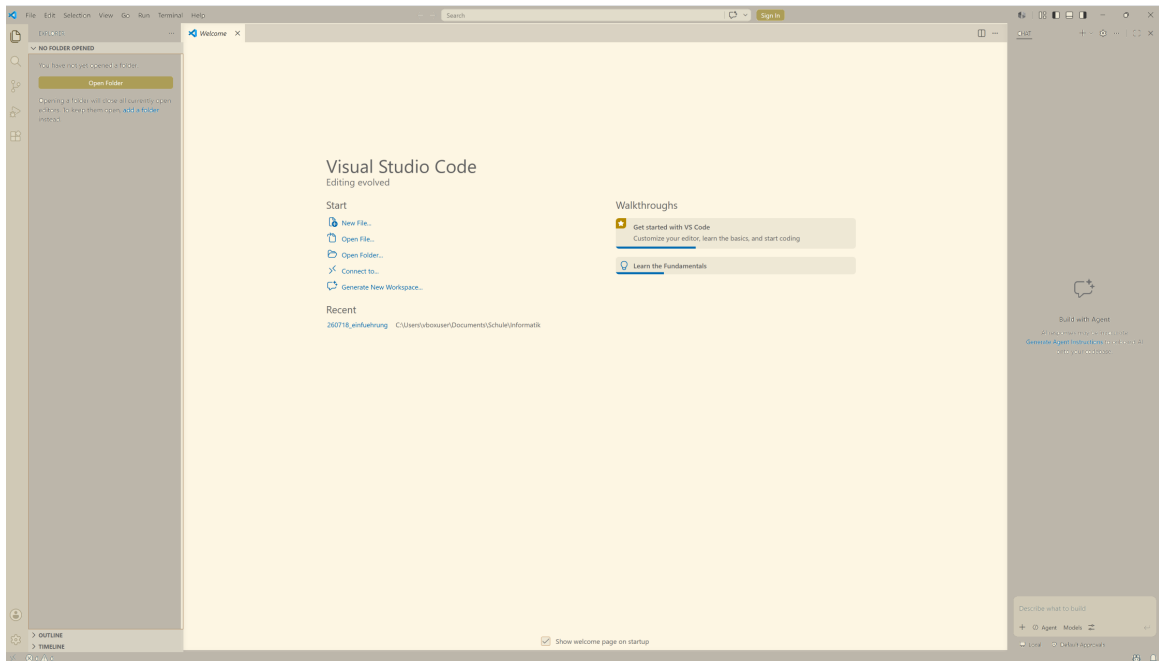


Abbildung 2.2.: VS Code Arbeitsbereich.

Links davon befindet sich die Navigationsleiste. Hier kann der neu angelegte Ordner als Arbeitsverzeichnis ausgewählt werden.

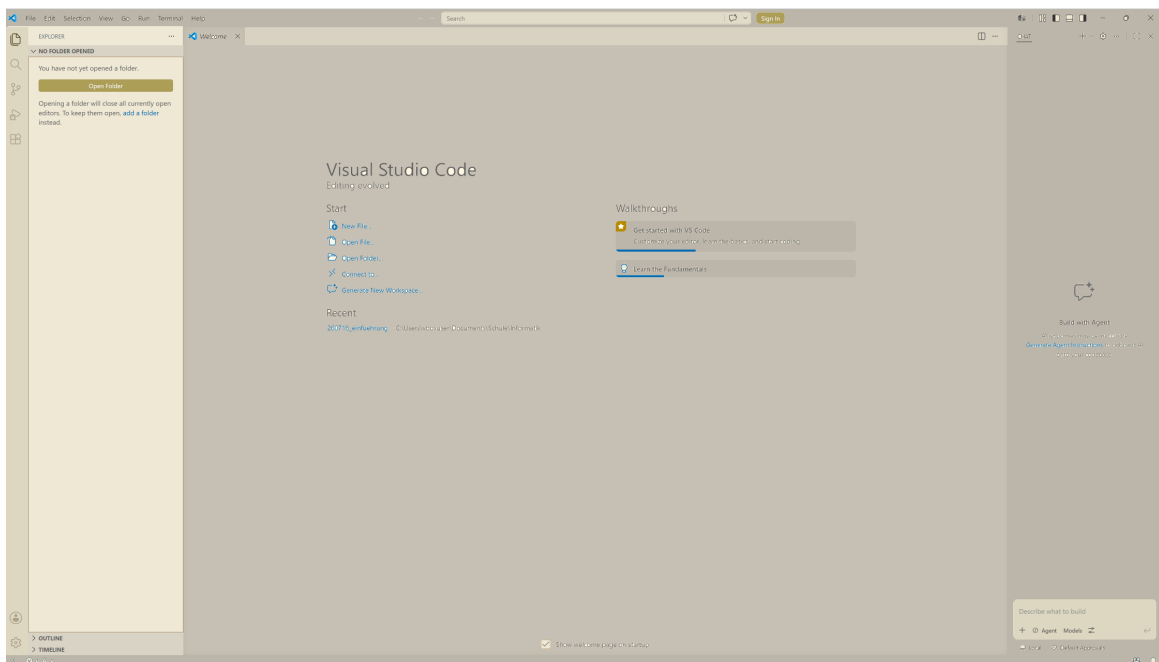


Abbildung 2.3.: VS Code Navigationsleiste.

Dazu muss man die Schaltfläche **Open Folder** anklicken und sich im Dialog bis zum gewünschten Ordner durchklicken. Wenn man im gewünschten Ordner angekommen ist, wird die Auswahl mit dem Schalter **Select folder** bestätigt.

Sobald der gewünschte Ordner ausgewählt worden ist, verändert sich das Aussehen des Fensters geringfügig.

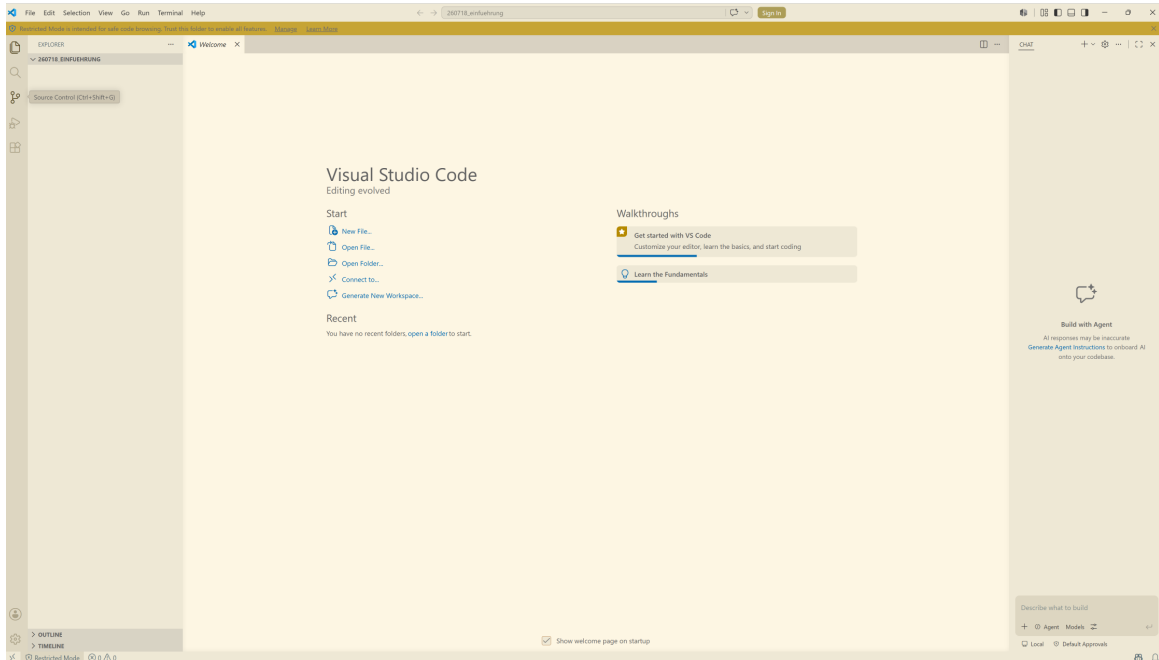


Abbildung 2.4.: VS Code mit ausgewähltem Arbeitsverzeichnis.

Die auffälligste Änderung ist der Balken mit dem Text 'Restricted Mode is intended for safe code browsing. Trust this folder to enable all features. Manage Learn More' zwischen Menüzeile und Arbeitsbereich. Klicken Sie auf **Manage** und wählen Sie im Dialog **Trust**. Anschliessend können Sie das Dialogfenster schliessen. Damit ist der Hinweisbalken verschwunden.

Eine zweite, etwas weniger auffällige Veränderung im Fenster ist der Titel des ausgewählten Ordners in der Navigationsleiste. Weil der Ordner im Moment noch leer ist, steht nur der Titel des Ordners da. Sobald wir eine Datei anlegen, erscheint diese auch in der Navigationsleiste. Bevor wir eine Datei anlegen aber noch der Hinweis auf den letzten bisher noch nicht besprochenen Teil des VS Code-Fensters.

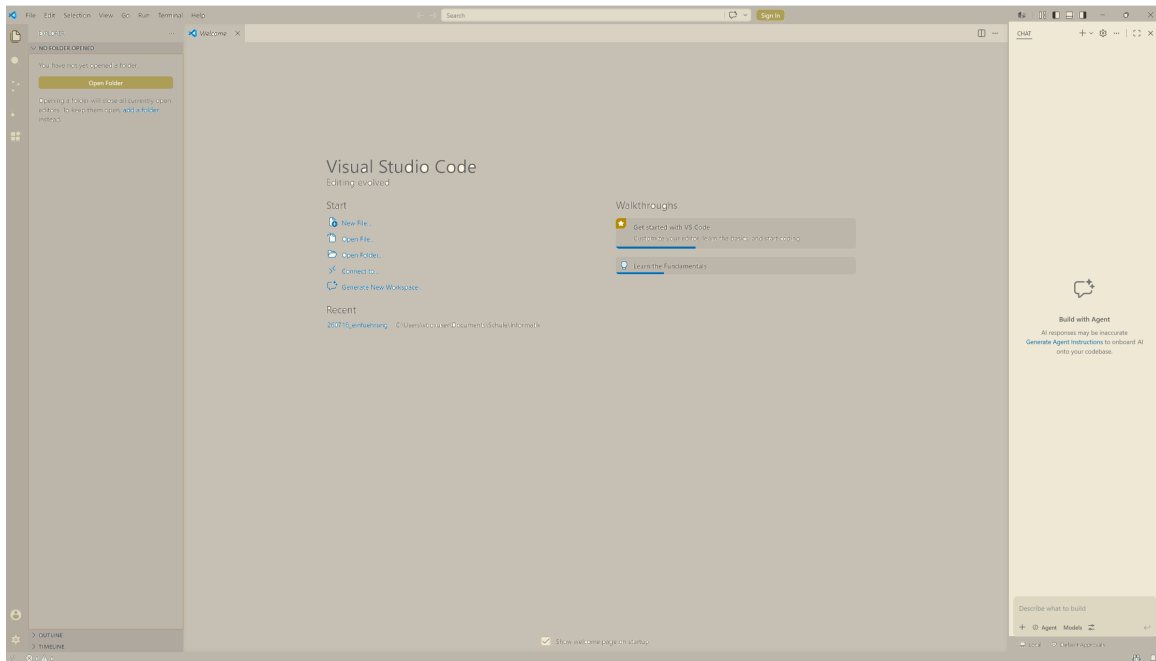


Abbildung 2.5.: VS Code AI Chat.

Microsoft als Herausgeber von VS Code will die Benutzer mit einem KI Chatbot unterstützen. Dabei handelt es sich um ein Freemium-Angebot; d.h. ein bescheidener Funktionsumfang steht gratis zur Verfügung und für den vollen Funktionsumfang muss man ein Abonnement lösen. Da wir im Informatikunterricht herausfinden wollen, wie Programme funktionieren, können wir diese Chat-Leiste wegklicken.

Um eine neue Datei anzulegen, fahren Sie mit der Maus in die Navigationsleiste. Dann erscheinen dort neben dem Namen des Ordners mehrere kleine Symbole. Das Symbol ganz links sieht aus, wie eine leere Seite Papier mit einem kleinen Plus an der unteren rechten Ecke. Dieses Symbol steht für das Anlegen einer neuen Datei. Wenn Sie das Symbol anklicken, öffnet sich in der Navigationsleiste ein Eingabefeld. Dort geben Sie den Namen der neuen Datei ein. Verwenden Sie den Namen `test.ipynb`. Dabei ist `test` der eigentliche Name der Datei und `.ipynb` die Dateinamenserweiterung für ein Jupyter Notebook. Damit erkennt VS Code, wie mit dieser Datei zu verfahren ist.

Das Fenster sollte jetzt wie Abbildung 2.6 aussehen.

Bevor Sie in diesem Notebook Python-Code ausführen können, müssen Sie mit der Schaltfläche **Select Kernel** oben rechts auswählen, wie VS Code Ihren Code ausführen soll. Der Kernel ist das Programm, das Ihren Python-Code im Notebook tatsächlich ausführt – VS Code muss einmalig pro Arbeitsverzeichnis wissen, welches Python es verwenden soll.

Wählen Sie...

1. ... Install Python and Jupyter.
2. ... Python Environments....

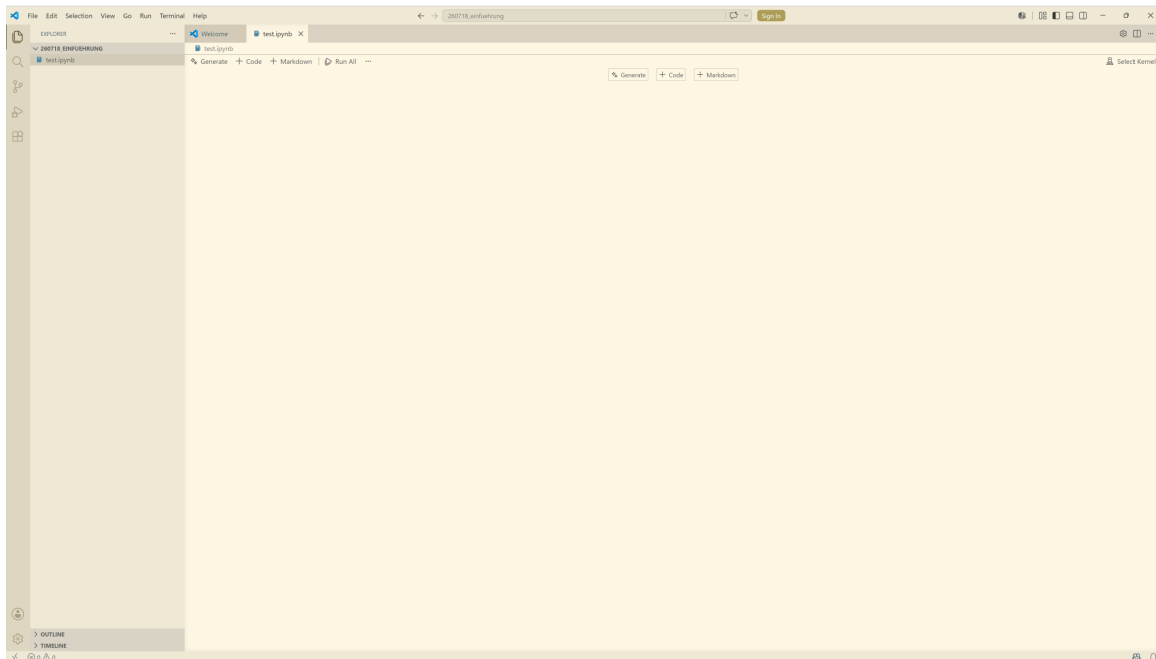


Abbildung 2.6.: VS Code mit einem offenen, aber leeren, Jupyter Notebook.

3. ... + Create Python Environment.
4. ... Quick Create.

Nach diesen Arbeitsschritten steht oben rechts nicht mehr **Select Kernel** sondern neu **.venv**

Jetzt können Sie mit der Schaltfläche **+ Code** eine sogenannte Code-Zelle öffnen. In dieser Zelle können Sie Ihren Python-Code schreiben. Damit Sie das Prüfen können, geben Sie in dieser Zelle `print('Hello World')` ein. Sie können die Code-Zelle ausführen, indem Sie auf den Play-Knopf am linken Rand der Zelle klicken oder indem Sie die Umschalttaste und die Enter Taste drücken.

Wenn Sie eine neue Zelle anlegen wollen, fahren Sie mit der Maus in der Mitte einer schon bestehenden Zelle entweder an den oberen oder unteren Rand der schon bestehenden Zelle. Sie können dann auswählen, ob Sie eine Code- oder eine Markdown-Zelle anlegen wollen. Markdown-Zellen sind für die Eingabe von Text vorgesehen. Sie können diesen Text mit Formatierungsbefehlen formatieren. Die entsprechenden Befehle finden Sie auf der Website von [Markdown Guide](#).

3. Eine sehr kurze Einführung in Python

In diesem Jupyter Notebook erhalten Sie einen ganz kurzen Überblick über die wichtigsten Elemente der Programmiersprache Python. Damit wird die Grundlage für die kommenden Themen gelegt.

Wenn Sie dieses Jupyter Notebook durchgearbeitet haben, sollten Sie in der Lage sein

- mit Python Berechnungen in den Grundrechenarten vorzunehmen;
- Variablen Werte zuzuweisen sowie
- einfache, von Python direkt zur Verfügung gestellte Funktionen aufzurufen.

3.1. Rechnen mit Python

Es gibt verschiedene Möglichkeiten mit Hilfe eines Computers arithmetische Operationen durchzuführen. Hier lernen Sie, wie Sie solche mit Python vornehmen können.

Python stellt Ihnen die folgenden arithmetischen Operationen zur Verfügung:

Tabelle 3.1.: Zusammenstellung der Grundrechenarten.

Operation	Operationszeichen	Algebraisch	Python	Numerisch
Addition	+	$a + b$	<code>4 + 5</code>	$4 + 5 = 9$
Subtraktion	−	$a - b$	<code>5 - 4</code>	$5 - 4 = 1$
Multiplikation	·	$a \cdot b$	<code>3 * 4</code>	$3 \cdot 4 = 12$
Potenzierung		a^b	<code>2 ** 3</code>	$2^3 = 8$
Division		$\frac{a}{b}$	<code>7 / 2</code>	$\frac{7}{2} = 3.5$
Ganzzahldivision	<code>[]</code>	$\lfloor \frac{a}{b} \rfloor$	<code>7 // 2</code>	$\lfloor \frac{7}{2} \rfloor = 3$
Modulo	<code>mod</code>	$a \bmod b$	<code>7 % 2</code>	$7 \bmod 2 = 1$

Python ist so programmiert, dass die Rangfolge der verschiedenen arithmetischen Operationen (Punkt vor Strich, Klammern zuerst) eingehalten wird.

Damit Sie ein Gefühl für die Funktionsweise von Python als ‘Rechner’ erhalten, führen Sie die folgenden Berechnungen durch. Die Berechnung kann direkt in der Code-Zelle unterhalb der jeweiligen Aufgabenstellung durchgeführt werden.

a) $2 + 3 \cdot 4$

```
# TODO: Berechnung durchführen
```

b) $(2 + 3) \cdot 4$

```
# TODO: Berechnung durchführen
```

c) $2^5 + 2^3 + 2^1$

```
# TODO: Berechnung durchführen
```

d) $\lfloor \frac{40}{7} \rfloor$

```
# TODO: Berechnung durchführen
```

e) $40 \bmod 7$

```
# TODO: Berechnung durchführen
```

3.2. Werte Variablen zuweisen

Die oben durchgeführten Berechnungen haben das Resultat jeweils direkt ausgegeben. In vielen Fällen ist das Resultat einer konkreten Berechnung jedoch lediglich ein Zwischenresultat. In diesen Fällen will man mit diesem Zwischenresultat weiterrechnen können. Hier kommen Variablen ins Spiel.

Die Resultate Ihrer Berechnungen werden nicht nur angezeigt, sie werden auch im Arbeitsspeicher abgelegt. Variablen dienen nun dazu, einen ansprechbaren Verweis auf den Ort des abgelegten Resultates anzulegen. Das heisst, wir können die Resultate der Berechnungen nicht nur ausgeben, wir können sie einer Variable zuweisen. Dazu wird in Python das Zeichen `=` verwendet (ich habe das Zeichen in diesem Zusammenhang bewusst nicht Gleichheitszeichen genannt, weil es hier keine Gleichheit, sondern eine Zuweisung zum Ausdruck bringt). Dabei muss die Variable links des Zuweisungszeichens und der zuzuweisende Wert rechts davon stehen.

Variablennamen dürfen aus Buchstaben, Ziffern und Unterstrichen bestehen, dürfen aber nicht mit einer Ziffer beginnen. Ausgenommen sind die sogenannten ‘[reservierten Wörter](#)’. Diese Wörter haben in Python eine feste Bedeutung und dürfen deshalb nicht als Variablennamen verwendet werden. Als Konvention hat sich eingebürgert, dass Python-Variablen in Snake Case geschrieben werden. Snake Case sieht folgendermassen aus: `ich_bin_eine_python_variable`. Dies ermöglicht es, Variablen mit Namen zu versehen, welche eine kurze Erklärung zu den ihnen zugewiesenen Werten abgeben.

Für die erste Berechnung von oben kann das folgendermassen aussehen:

```
resultat_a = 2 + 3 * 4
```

Nun wird das Resultat der Berechnung der Variable `resultat_a` zugewiesen und nicht mehr ausgegeben. Wir können den zugewiesenen Wert jederzeit ausgeben, indem wir die Variable aufrufen.

```
resultat_a
```

Die Berechnung und der Aufruf der Variable können auch in der gleichen Code-Zelle stehen. Ausgegeben wird allerdings immer nur der Wert der letzten Code-Zeile.

```
resultat_a = 2 + 3 * 4  
resultat_a
```

Spielen Sie die verschiedenen Möglichkeiten mit allen obigen Berechnungen durch.

```
# TODO: Resultate von Berechnungen Variablen zuweisen
```

Variablen können jedoch nicht ausschliesslich Werte von Berechnungen zugewiesen werden. Es ist auch möglich, Variablen direkt Werte zuzuweisen. Ausserdem kann mit den Variablen weitergerechnet werden. Das zeigt das folgende Beispiel.

```
a = 3  
b = 4  
c = (a**2 + b**2)**(1/2)  
c
```

5.0

Diese Berechnung ermöglicht noch eine Ergänzung zu den oben in der Tabelle angeführten arithmetischen Operationen. Wurzeln können als Potenz dargestellt werden. Für die Quadratwurzel ist der Exponent ein Zweitel, für die dritte Wurzel ein Drittel etc. Dabei dürfen die Exponenten auch als Dezimalzahl geschrieben werden.

Führen Sie die obige Berechnung mit den Zahlen $a = 5$ und $b = 12$ durch.

```
# TODO: Berechnung mit a = 5 und b = 12 durchführen
```

3.3. Einfache Python Funktionen aufrufen

Die erste Funktion, welche bei der Neuinstallation einer Programmiersprache traditionellerweise aufgerufen wird, haben Sie bereits kennengelernt.

```
print('Hello World')
```

Die Funktion `print()` gibt aus, was ihr innerhalb der Klammern übergeben wird. Was einer Funktion beim Aufruf übergeben wird, nennt man Argument. So, wie wir die Funktion bisher verwendet haben, ist das Argument die Zeichenfolge in den Anführungszeichen. Anstelle eines solchen Strings (Zeichenkette, ein von Python zur Verfügung gestellter Datentyp - was genau Datentypen sind, wird gleich unten angetönt und später im Detail besprochen) können der Funktion auch Variablen übergeben werden. So kann ihr zum Beispiel die Variable `resultat_a` aus obigen Berechnungen übergeben werden.

```
print(resultat_a)
```

Hinweis: Erscheint hier die Fehlermeldung `NameError: name 'resultat_a' is not defined`, wurde die Zelle, in der die Variable definiert wird, in dieser Sitzung noch nicht ausgeführt. Führen Sie zuerst die Zelle mit der Zuweisung aus und anschliessend diese Zelle noch einmal.

Der Unterschied zum direkten Aufruf einer Variable: Ohne `print()` zeigt das Notebook nur den Wert der letzten Zeile einer Code-Zelle an. Mit `print()` können Sie beliebig viele Werte an beliebiger Stelle einer Code-Zelle ausgeben. Probieren Sie das aus, indem Sie in der folgenden Code-Zelle alle von Ihnen bisher verwendeten Variablen nacheinander mit `print()` ausgeben.

```
# TODO: Ausgabe der bisher definierten Variablen
```

Eine zweite nützliche Funktion, die Python direkt zur Verfügung stellt, ist `type()`. Sie verrät, welchen Datentyp ein Wert hat. Wie bei `print()` wird der zu untersuchende Wert als Argument in den Klammern übergeben.

```
type(7)
```

Die Ausgabe `int` steht für Integer, den Datentyp der ganzen Zahlen. Untersuchen Sie in der folgenden Code-Zelle nacheinander die Werte `3.5`, `'Hello World'` sowie die Variablen `resultat_a` und `c`. Verwenden Sie dazu `print()` in Kombination mit `type()`, also zum Beispiel `print(type(3.5))` - so werden alle Resultate untereinander ausgegeben.

Sie werden dabei neben `int` auf zwei weitere Datentypen stossen: `float` (Dezimalzahlen, sogenannte Fließkommazahlen) und `str` (Strings, also Zeichenketten). Was es mit diesen Datentypen im Einzelnen auf sich hat, wird später im Detail besprochen. Für den Moment genügt es zu wissen, dass jeder Wert in Python einen Datentyp hat und dass `type()` diesen sichtbar macht.

```
# TODO: Datentypen der angegebenen Werte und Variablen ausgeben
```

4. Problemlösung in der Informatik

4.1. Ausgangslage

Jede Disziplin hat ihre eigene Art, Probleme zu lösen. Das ist in der Informatik nicht anders.

In der Informatik versucht man, grosse Probleme in kleinere Teilprobleme zu zerlegen. Das macht man so lange, bis die Teilprobleme klein genug sind, damit sie einfach zu lösen sind.

Diese Vorgehensweise wird hier geübt, indem Wege gesucht werden, Muster nachzubilden. Um die Muster in Python darstellen zu können, verwenden wir die Python Bibliothek **PyTamaro**. Diese Bibliothek wurde von der Università della Svizzera italiana (USI) extra für den Informatik-Unterricht entwickelt.

Python-Libraries (Bibliotheken)

Programmierfunktionen, welche häufig gebraucht werden, von Python aber nicht standardmässig zur Verfügung gestellt werden, werden oft in sogenannten Libraries bereitgestellt.

Es kann sein, dass eine solche Library erst installiert werden muss. Dies geschieht im Terminal mit dem Befehl `python -m pip install <name_der_library>`. Damit die Library in einem Jupyter Notebook verwendet werden kann, muss sie in einer Code Zelle mit

```
import name_der_library
```

importiert werden.

Wenn nur einzelne Funktionen einer Library verwendet werden sollen, lautet der Befehl für den Import

```
from name_der_library import name_der_funktion
```

Bevor die **PyTamaro**-Bibliothek verwendet werden kann, muss sie im Terminal mit `python -m pip install pytamaro` installiert werden.

Aus der **PyTamaro**-Bibliothek brauchen wir für die ersten Übungen nur einen Teil der Funktionen. Diese werden daher in der folgenden Code-Zelle einzeln importiert.

```
from pytamaro.de import (  
    rechteck, kreis_sektor,  
    blau, rot, weiss, schwarz,  
    neben, ueber, ueberlagere,  
    drehe, kombiniere,  
    zeige_grafik,  
)
```

4.2. Beispiel: Tricolore

Die Vorgehensweise für die Problemlösung in der Informatik wird am Beispiel der Französischen Nationalflagge (Tricolore) gezeigt.

! Illustrationen in Markdown

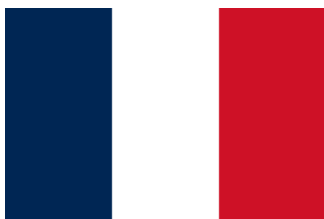
Wie in Kapitel 2 am Ende beschrieben, werden die Textblöcke in Jupyter Notebooks mit Markdown formatiert. Die Syntax zum Einbinden von Illustrationen in einen mit Markdown formatierten Text lautet

! [Alternativtext] (pfad/zur/illustration)

Solange der entsprechende Text nur lokal, das heisst auf dem gleichen Computer, weiterverarbeitet wird, auf dem er geschrieben wurde, ist das kein Problem. Wenn der Text jedoch weitergegeben wird, müssen auch die Dateien mit den Illustrationen mitgegeben werden. Die Schwierigkeit dabei ist, dass bei der Weitergabe die Ordnerstruktur erhalten werden muss. Wenn das nicht sichergestellt ist, können die Illustrationen nicht dargestellt werden.

Um diese Schwierigkeit zu umgehen, können Illustrationen im sogenannten Base64-Format dargestellt werden. Dieses Format ist eine zusammenhängende Folge von Zahlen und Buchstaben (das Verfahren wird später besprochen). Diese Zeichenfolge kann anstelle des Pfades zur Illustration in der Klammer zum Einfügen der Illustration eingesetzt werden. So kann die Illustration innerhalb des Jupyter Notebooks gespeichert werden.

Erschrecken Sie also nicht ob der zum Teil sehr langen Zeichenfolge in den Markdown-Blöcken mit Illustrationen.



Um die hier abgebildete Tricolore zu rekonstruieren, wird die Grafik in ihre Einzelteile zerlegt.

Die Tricolore hat insgesamt ein Seitenverhältnis von 3:2 (das heisst, sie ist drei Einheiten breit und zwei Einheiten hoch). Innerhalb des so definierten Rechtecks besteht sie aus drei gleich grossen Rechtecken in den Farben blau, weiss und rot mit einem Seitenverhältnis von 1:2 (Breite zu Höhe). Die drei farbigen Rechtecke sind nebeneinander angeordnet.

Die von PyTamaro in Python für das Erstellen von Rechtecken zur Verfügung gestellte Funktion hat die folgende Syntax:

```
name = rechteck(breite, höhe, farbe)
```

Bevor die Zeichnung tatsächlich erstellt wird, soll hier der Befehl im Detail erklärt werden:

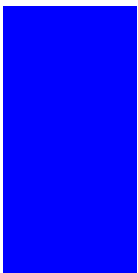
name ist der Name, unter dem das Rechteck gespeichert wird. Dieser Name kann später verwendet werden, um auf das Rechteck zuzugreifen. **rechteck** ist die Funktion, welche ein Rechteck zeichnet. In der Klammer hinter dem Befehl werden die sogenannten Argumente angegeben. Diese steuern, wie das Rechteck aussieht. **breite** und **höhe** sind die Argumente, die die Grösse des Rechtecks bestimmen. Diese Werte können beliebig gewählt werden. **farbe** ist das Argument, das die Farbe des Rechtecks bestimmt. Aufgrund der Eigenheiten von PyTamaro können ausschliesslich die Farben verwendet werden, welche zuvor importiert worden sind.

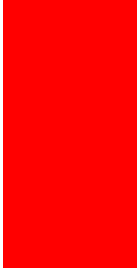
Die drei Rechtecke der Tricolore werden damit mit der folgenden Befehlssequenz erstellt:

```
bleu = rechteck(50, 100, blau)
blanc = rechteck(50, 100, weiss)
rouge = rechteck(50, 100, rot)
```

Um die so erstellten Rechtecke anzeigen zu können, brauchen wir die Funktion `zeige_grafik(name_der_anzuzeigenden_grafik)`.

```
zeige_grafik(bleu)
zeige_grafik(blanc)
zeige_grafik(rouge)
```

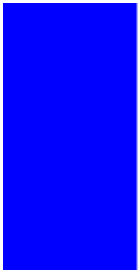




Als nächstes werden die drei Rechtecke nebeneinander angeordnet. Dazu wird die Funktion `neben(linker_grafik, rechter_grafik)` verwendet. Die beiden Argumente, die der Funktion übergeben werden, sind die linke und die rechte Grafik, die zusammengefügt werden sollen.

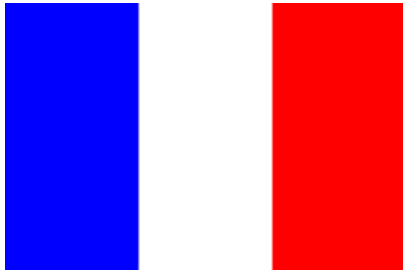
So können die ersten zwei Drittel der Tricolore erstellt werden:

```
deux_tiers = neben(bleu, blanc)
zeige_grafik(deux_tiers)
```



Um die Tricolore zu vervollständigen, müssen nun noch die bestehenden zwei Drittel mit dem roten Rechteck verbunden werden.

```
tricolore = neben(deux_tiers, rouge)
zeige_grafik(tricolore)
```



4.3. Aufgabe: Österreichische Flagge

Übertragen Sie das obige Beispiel auf die Österreichische Flagge. Das Seitenverhältnis der Österreichischen Flagge ist 2:3.



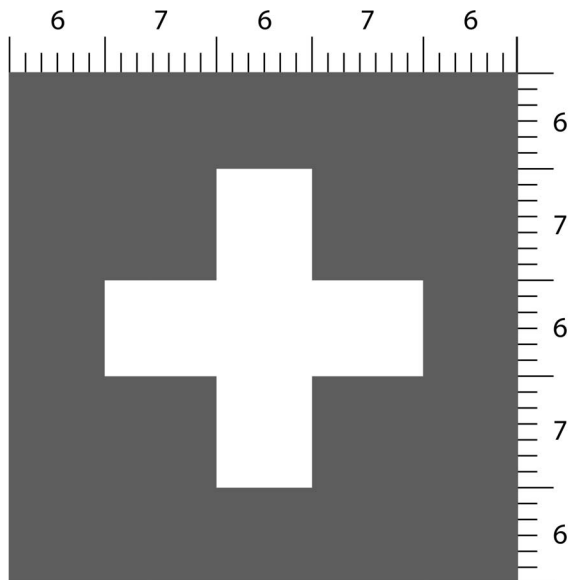
PyTamaro stellt eine Funktion zur Verfügung, mit welcher einzelne Grafiken übereinander angeordnet werden können. Die Syntax dieser Funktion lautet:

```
resultat = ueber(obere_grafik, untere_grafik)
```

```
# TODO: Erstellen Sie hier eine Österreichische Flagge
```

4.4. Aufgabe: Schweizerfahne

Zeichnen Sie eine korrekt proportionierte Schweizerfahne. Die Dimensionen gemäss [Art. 3 Abs. 2 i.V.m. Anhang 2 WSchG](#) finden sie in der folgenden Abbildung.



Neben den Funktionen `rechteck()`, `neben()` und `ueber()` stellt **PyTamaro** die Funktionen `drehe()` und `ueberlagere()` zur Verfügung. Für die vollständige Syntax wird an dieser Stelle auf das [Manual](#) verwiesen. Die für die folgenden Aufgaben wichtigsten Funktionen haben die folgende Syntax:

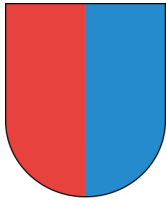
```
resultat = ueber(obere_grafik, untere_grafik)
resultat = ueberlagere(vordere_grafik, hintere_grafik)
resultat = drehe(winkel, grafik)
resultat = kreis_sektor(radius, winkel, farbe)
```

Bei `drehe()` wird der Winkel in Grad angegeben und gegen den Uhrzeigersinn gedreht. Bei `ueberlagere()` wird die erste Grafik vor die zweite gelegt (beide werden in ihren Mittelpunkten zentriert).

```
# TODO: Zeichnen Sie eine Schweizerfahne
```

4.4.1. Aufgabe: Tessiner Wappen

Als Referenz an die USI und den Kanton Tessin zeichnen Sie bitte das Tessiner Wappen gemäss der folgenden Vorlage.



Verwenden Sie dazu neben den bereits bekannten Funktionen zusätzlich die Funktion `kreis_sektor()`.

```
# TODO: Zeichnen Sie das Tessiner Wappen
```

4.5. Musterlösungen

4.5.1. Österreichische Flagge

```
roter_streifen = rechteck(135, 30, rot)
weisser_streifen = rechteck(135, 30, weiss)
zwei_drittel = ueber(roter_streifen, weisser_streifen)
oesterreich = ueber(zwei_drittel, roter_streifen)
zeige_grafik(oesterreich)
```



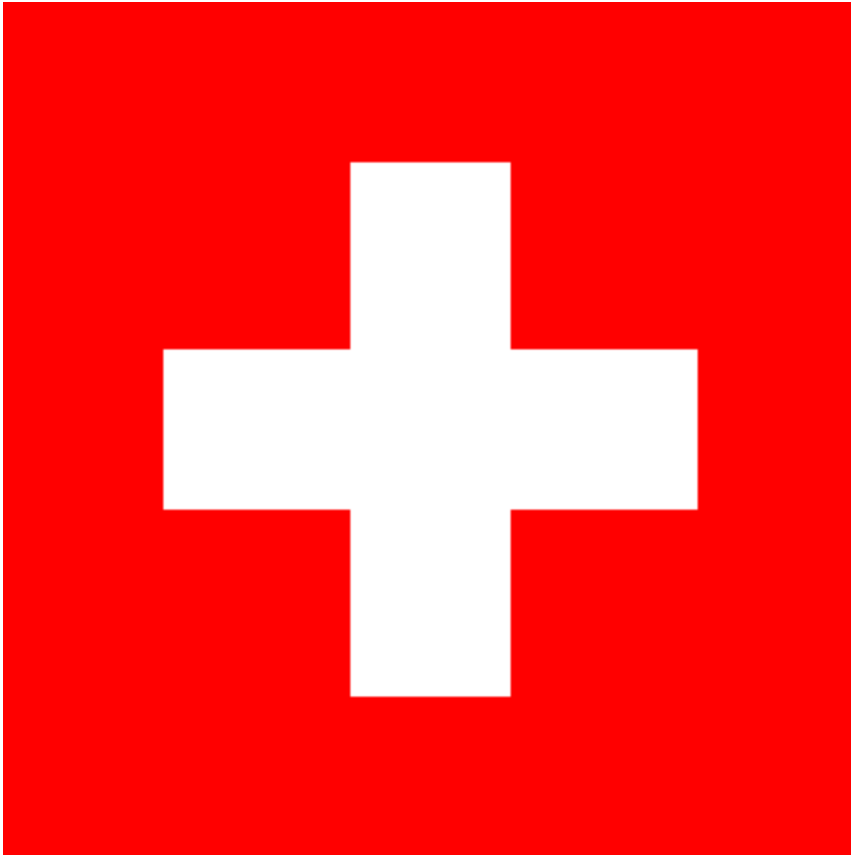
4.5.2. Schweizerfahne

```
# Die Armbreite des Kreuzes beträgt rund 1/5 der Fahnenhöhe,
# die Armlänge das 7/6-fache der Armbreite. Bei einer Balkenbreite
# von 60 ergibt das eine Gesamtlänge des Balkens von 60 + 2 * 70 = 200.
hintergrund = rechteck(320, 320, rot)
weisser_balken_horizontal = rechteck(200, 60, weiss)
weisser_balken_vertikal = drehe(90, weisser_balken_horizontal)

kreuz = ueberlagere(weisser_balken_horizontal, weisser_balken_vertikal)

schweizerfahne = ueberlagere(kreuz, hintergrund)
```

```
zeige_grafik(schweizerfahne)
```



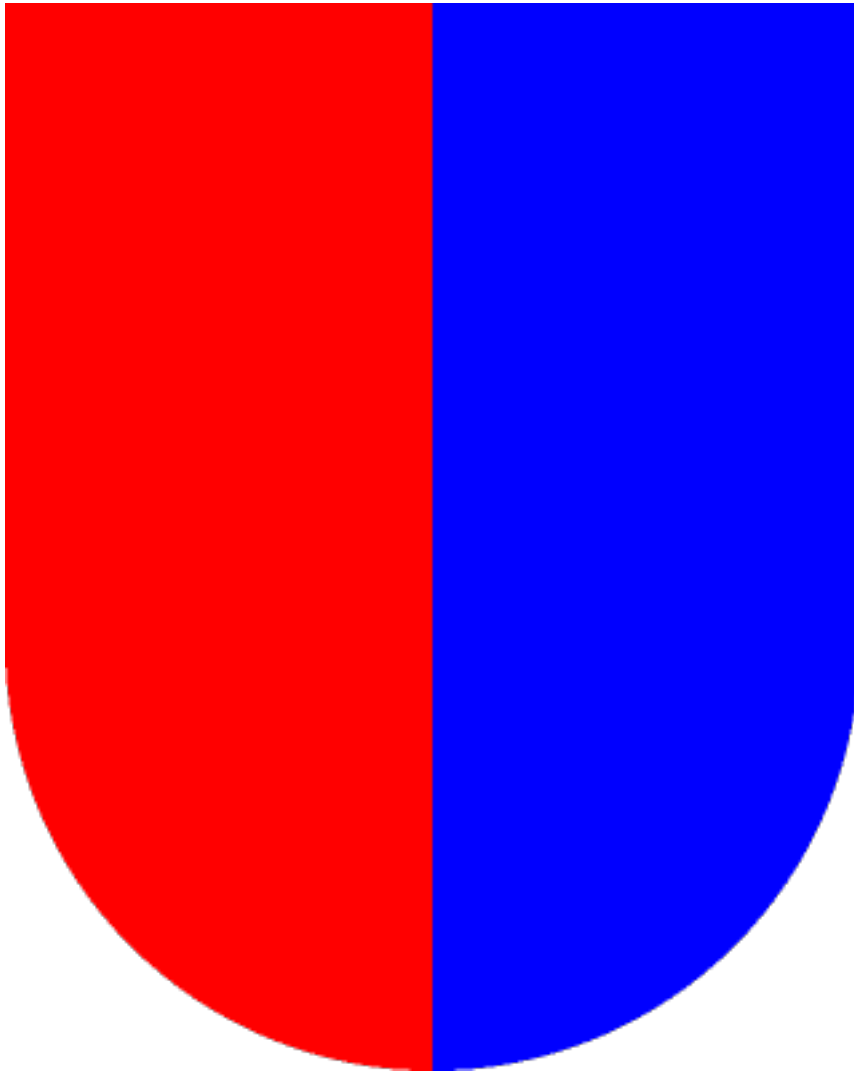
4.5.3. Tessin

```
rotes_rechteck = rechteck(160, 240, rot)
blaues_rechteck = rechteck(160, 240, blau)
roter_sektor = kreis_sektor(160, 90, rot)
roter_sektor_gedreht = drehe(180, roter_sektor)
blauer_sektor = kreis_sektor(160, 90, blau)
blauer_sektor_gedreht = drehe(270, blauer_sektor)

ti_unten = neben(roter_sektor_gedreht, blauer_sektor_gedreht)
ti_oben = neben(rotes_rechteck, blaues_rechteck)

tessin = ueber(ti_oben, ti_unten)

zeige_grafik(tessin)
```



5. Wiederholungen in Python (For-Schleifen)

Eine Stärke von Computerprogrammen ist die wiederholte Ausführung von Anweisungen. Viele Programmiersprachen stellen dafür ein Konstrukt mit dem Namen 'For-Schleife' zur Verfügung. Eine 'For-Schleife' funktioniert unabhängig von einer konkreten Programmiersprache folgendermassen:

```
FÜR variable VON startwert BIS endwert [MIT schrittweite]
    Anweisungen
ENDE FÜR
```

Übersetzt nach Python sieht das so aus:

```
for i in range(n):
    do...
```

`startwert BIS endwert [MIT schrittweite]` wird dabei durch `range(n)` ausgedrückt. Dabei ist `n` der Endwert. Gezählt wird bis zum aber ohne den Endwert. Startwert und Schrittweite haben Vorgabewerte, d.h., sie müssen nicht explizit angegeben werden. Der Vorgabewert für den Start ist 0, derjenige der Schrittweite 1. Wenn der Startwert abweichend vom Vorgabewert festgelegt werden soll, kann dieser explizit angegeben werden. Der Aufruf von `range()` sieht dann so aus:

```
range(startwert, endwert)
```

Falls eine von 1 abweichende Schrittweite festgelegt werden soll lautet der Aufruf

```
range(startwert, endwert, schrittweite)
```

In diesem Fall müssen neben dem Endwert sowohl der Startwert und die Schrittweite angegeben werden. Andernfalls kann nicht zwischen den einzelnen Angaben zu Endwert, Startwert und Schrittweite unterschieden werden.

In den folgenden Code-Zellen finden Sie je ein technisches Beispiel für den Aufruf einer For-Schleife in Python.

```
# Grundbeispiel: Ziffern von 0 bis 9
for i in range(10):
    print(i)
```

```
# Beispiel mit Startwert: Zählen von 1 bis 10
for i in range(1, 11):
    print(i)
```

```
# Beispiel mit Startwert und Schrittweite: Zweierreihe
for i in range(2, 21, 2):
    print(i)
```

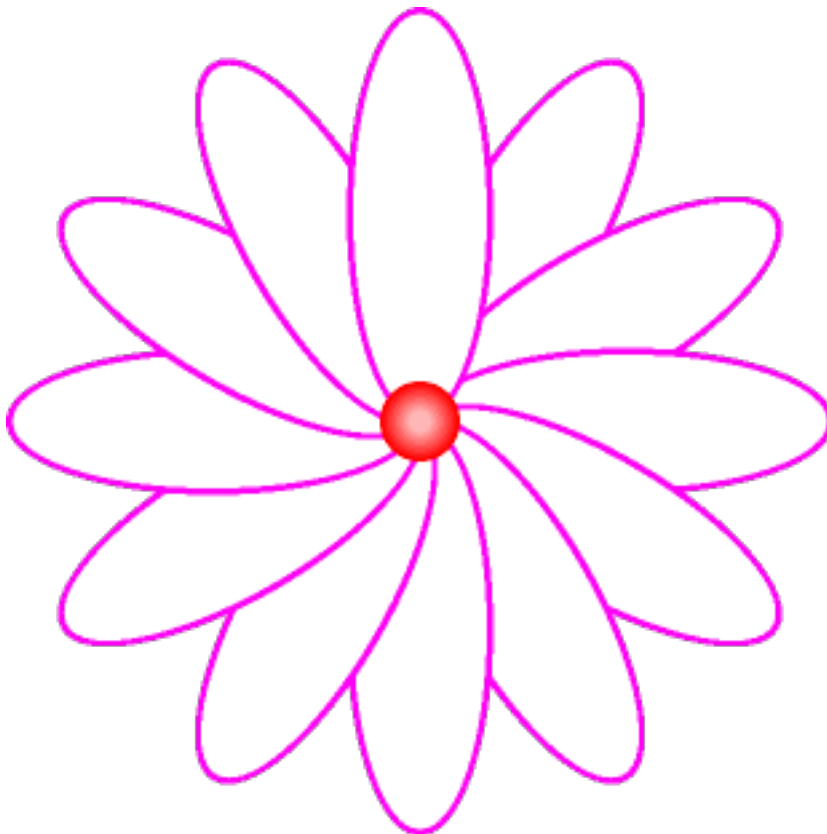
5.1. Anwendungsübung zu Wiederholungen in Python mit PyTamaro

Im Folgenden bauen Sie eine Blume mit Python For-Schleifen und PyTamaro.

Dafür importieren wir in der nächsten Zelle alle deutschsprachigen Funktionen, welche PyTamaro zur Verfügung stellt. Eine Beschreibung der Funktionen mit der genauen Syntax finden Sie in der [PyTamaro Dokumentation](#).

```
from pytamaro.de import * # der Stern steht dabei als Platzhalter für alles
```

Das Ziel ist es, die untenstehende Blume nachzubauen.



Die wichtigsten Funktionen von PyTamaro für diese Aufgabe sind `fixiere()` und `kombiniere()`.

Die Funktion `fixiere()` legt für eine Grafik einen frei gewählten Ankerpunkt fest. Dazu muss man wissen, dass jede in PyTamaro gezeichnete Grafik von einer sogenannten “Bounding Box” umgeben ist. Diese Bounding Box ist ein Rechteck, das die Grafik vollständig umschliesst. Standardmässig liegt der Ankerpunkt einer Grafik in der Mitte der Bounding Box. Mit `fixiere()` kann man den Ankerpunkt jedoch an eine andere Stelle der Bounding Box verschieben. PyTamaro stellt die folgenden Positionen für Ankerpunkte zur Verfügung:

- `mitte`: Mitte der Bounding Box (Standard)
- `mitte_links`: Mitte der linken Seite der Bounding Box
- `mitte_rechts`: Mitte der rechten Seite der Bounding Box
- `oben_links`: Obere linke Ecke der Bounding Box
- `oben_mitte`: Mitte der oberen Seite der Bounding Box
- `oben_rechts`: Obere rechte Ecke der Bounding Box
- `unten_links`: Untere linke Ecke der Bounding Box
- `unten_mitte`: Mitte der unteren Seite der Bounding Box
- `unten_rechts`: Untere rechte Ecke der Bounding Box

Der Ankerpunkt wird mit dem Befehl `fixiere(position, grafik)` gesetzt.

Der Ankerpunkt ist wichtig für die Funktion `drehe()`, die eine Grafik um einen bestimmten Winkel dreht. Die Drehung erfolgt immer um den Ankerpunkt.

Die Funktion `kombiniere(vordere Grafik, hintere Grafik)` fügt zwei Grafiken zusammen. Die erste gegebene Grafik liegt im Vordergrund und die zweite im Hintergrund. Die Grafiken werden so ausgerichtet, dass ihre Ankerpunkte übereinanderliegen.

5.1.1. Schritt 1: Zerlege das Bild in seine Einzelteile

Die Blume besteht im wesentlichen aus zwei Formen:

1. Blütenblätter in Form von Ellipsen sowie
2. einer Scheibe in der Form eines Kreises in der Mitte.

Um die Blume zu zeichnen, müssen diese beiden Formen zuerst einzeln erstellt werden.

```
bluetenblatt = ellipse(50, 150, blau)
# die Farbe blau wurde gewählt, damit das Blatt vor dem Hintergrund
# sichtbar ist

zeige_grafik(bluetenblatt)
```



```
scheibe = ellipse(30, 30, rot)
zeige_grafik(scheibe)
```



5.1.2. Schritt 2: Positioniere die Blütenblätter

Die Blütenblätter sind rund um die Mitte der Blume angeordnet. Da es zwölf Blütenblätter sind, beträgt der Winkel zwischen zwei Blütenblättern $\frac{360^\circ}{12} = 30^\circ$. Die Drehung der Blütenblätter erfolgt um den Ankerpunkt `unten_mitte` der Blütenblätter.

Die Blütenblätter müssen also zuerst fixiert und dann gedreht werden.

Allgemein gilt: Bei `anzahl_blaetter` gleichmässig verteilten Blütenblättern beträgt der Winkel zwischen zwei benachbarten Blättern $\frac{360^\circ}{\text{anzahl_blaetter}}$. Wer die Anzahl der Blätter ändert, muss deshalb sowohl den Bereich der Schleife als auch den Drehwinkel anpassen. Am einfachsten führt man dazu eine Variable `anzahl_blaetter` ein und berechnet den Winkel daraus.

```
bluetenblatt_fixiert = fixiere(unten_mitte, bluetenblatt)
bluetenblatt_30 = drehe(30, bluetenblatt_fixiert)
zeige_grafik(bluetenblatt_30)
```



5.1.3. Schritt 3: Zeichnen aller erforderlichen Blütenblätter

Die Blume hat zwölf Blütenblätter. Diese können von Hand jedes einzelne erstellt werden. Das dritte Blütenblatt ist um 60° gedreht und wird entsprechend mit

```
bluetenblatt_60 = drehe(60, bluetenblatt_fixiert)
```

erstellt. Das kann man für alle zwölf Blütenblätter machen bis man beim zwölften Blütenblatt angekommen ist, das um 330° gedreht ist.

```
bluetenblatt_330 = drehe(330, bluetenblatt_fixiert)
```

Das ist aber sehr mühsam und fehleranfällig. Viel einfacher ist es, eine Schleife (for-loop) zu verwenden, die die Blütenblätter basierend auf der Grundform automatisch erstellt.

```
anzahl_blaetter = 12

for i in range(anzahl_blaetter):
    winkel = i * 360 / anzahl_blaetter
    bluetenblatt_gedreht = drehe(winkel, bluetenblatt_fixiert)

zeige_grafik(bluetenblatt_gedreht)
```



Diese Schleife durchläuft bei `anzahl_blaetter = 12` die Zahlen von 0 bis 11 (also 12 Zahlen) und berechnet für jede Zahl mit `i * 360 / anzahl_blaetter` den entsprechenden Drehwinkel. Dann wird das Blütenblatt um diesen Winkel gedreht. Allerdings wird das gedrehte Blütenblatt in jeder Iteration der Schleife in der gleichen Variable `bluetenblatt_gedreht` gespeichert. Am Ende der Schleife enthält diese Variable nur das letzte gedrehte Blütenblatt (das um 330° gedrehte Blütenblatt).

5.1.4. Schritt 4: Kombiniere alle Blütenblätter

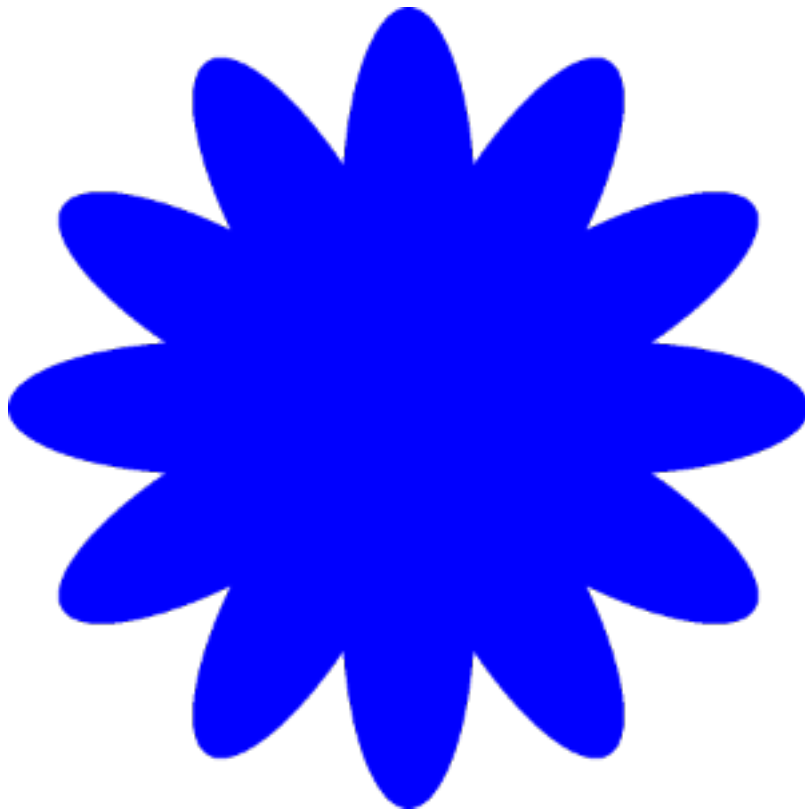
Um das unerwünschte Verhalten zu vermeiden, dass am Ende der Schleife nur das letzte gedrehte Blütenblatt in der Variable `bluetenblatt_gedreht` enthalten ist, müssen die gedrehten Blütenblätter in jeder Iteration der gewünschten Gesamtgrafik hinzugefügt werden. Dazu wird die Funktion `kombiniere()` verwendet.

```
anzahl_blaetter = 12

blume = bluetenblatt_fixiert

for i in range(anzahl_blaetter):
    winkel = i * 360 / anzahl_blaetter
    bluetenblatt_gedreht = drehe(winkel, bluetenblatt_fixiert)
    blume = kombiniere(blume, bluetenblatt_gedreht)

zeige_grafik(blume)
```



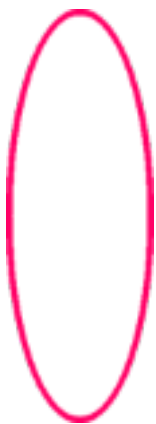
Damit sind die Blütenblätter in der gewünschten Position und Anzahl zusammengefügt. Damit Sie allerdings aussehen, wie die Blume in der Vorlage, müssen die einzelnen Blütenblätter weiss eingefärbt und mit einem rosa Rand versehen werden.

Rosa ist keine vordefinierte Farbe in PyTamaro. Sie können jedoch eine beliebige Farbe mit der Funktion `rgb_farbe(rot, gruen, blau)` erstellen. Dabei sind `rot`, `gruen` und `blau` Zahlenwerte von 0 bis 255, die den Anteil der jeweiligen Farbe an der Gesamtfarbe angeben. Damit nicht alle Kombinationen durchprobiert werden müssen, können Sie mit einem Online-Tool wie [RGB Color Picker](#) die gewünschte Farbe auswählen und die entsprechenden Werte für rot, grün und blau ablesen.

```
weisses_blatt = ellipse(50, 150, weiss)
rosa_rand = ellipse(55, 155, rgb_farbe(255, 0, 102))

blutenblatt_mit_rand = ueberlagere(weisses_blatt, rosa_rand)

zeige_grafik(blutenblatt_mit_rand)
```



Anschliessend können die Blütenblätter mit Rand in der vorher geschriebenen Schleife zur Gesamtgrafik zusammengefügt werden.

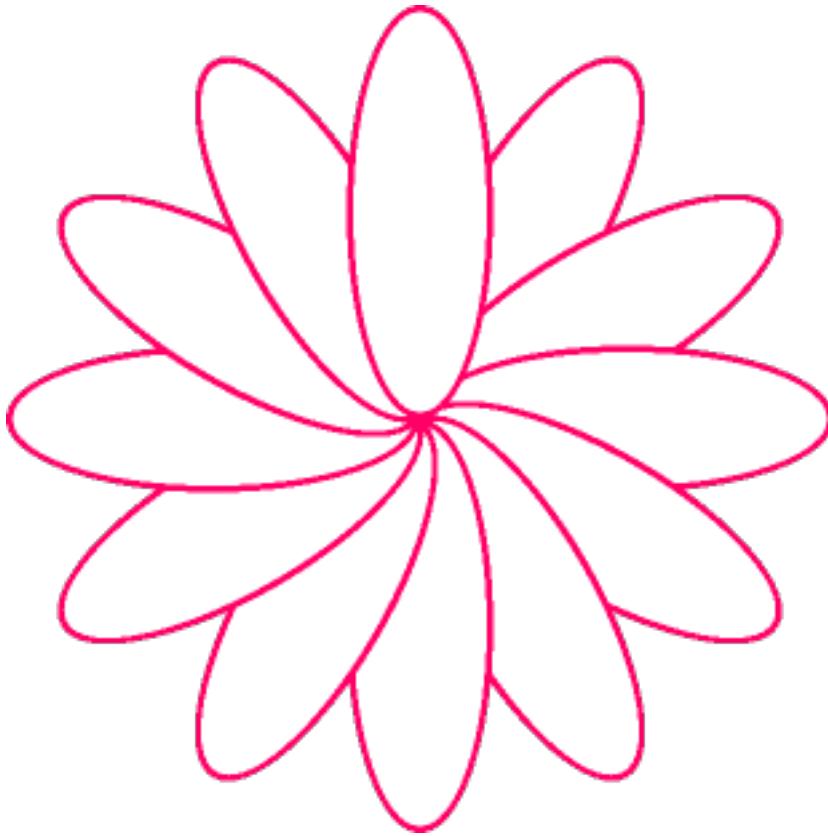
```
blutenblatt_mit_rand_fixiert = fixiere(unten_mitte, blutenblatt_mit_rand)

anzahl_blaetter = 12

blume_weiss = blutenblatt_mit_rand_fixiert

for i in range(anzahl_blaetter):
    winkel = i * 360 / anzahl_blaetter
    blutenblatt_gedreht = drehe(winkel, blutenblatt_mit_rand_fixiert)
    blume_weiss = kombiniere(blume_weiss, blutenblatt_gedreht)

zeige_grafik(blume_weiss)
```



5.1.5. Schritt 5: Scheibe im Zentrum der Blume

Bei genauer Betrachtung der Blume fällt auf, dass die Scheibe in der Mitte der Blume nicht einfach einfarbig rot ist, sondern gegen die Mitte hin einen Farbverlauf aufweist. Dieser Farbverlauf wird erzeugt, indem mehrere Kreise mit kleiner werdendem Radius und abnehmender Farbintensität übereinandergelegt werden.

```
# Start: satter roter Kreis (255, 0, 0).
# In jeder Iteration wird ein kleinerer Kreis darübergelegt, dessen
# Grün- und Blau-Anteil steigt (i*26) -> die Farbe wird zur Mitte hin
# heller (Richtung Weiss). Der Radius (30 - 3*i) muss positiv bleiben,
# deshalb läuft die Schleife nur bis 7.
scheibe = ellipse(30, 30, rgb_farbe(255, 0, 0))

for i in range(1, 8):
    tmp = ellipse(30 - 3*i, 30 - 3*i, rgb_farbe(255, i*26, i*26))
    scheibe = ueberlagere(tmp, scheibe)

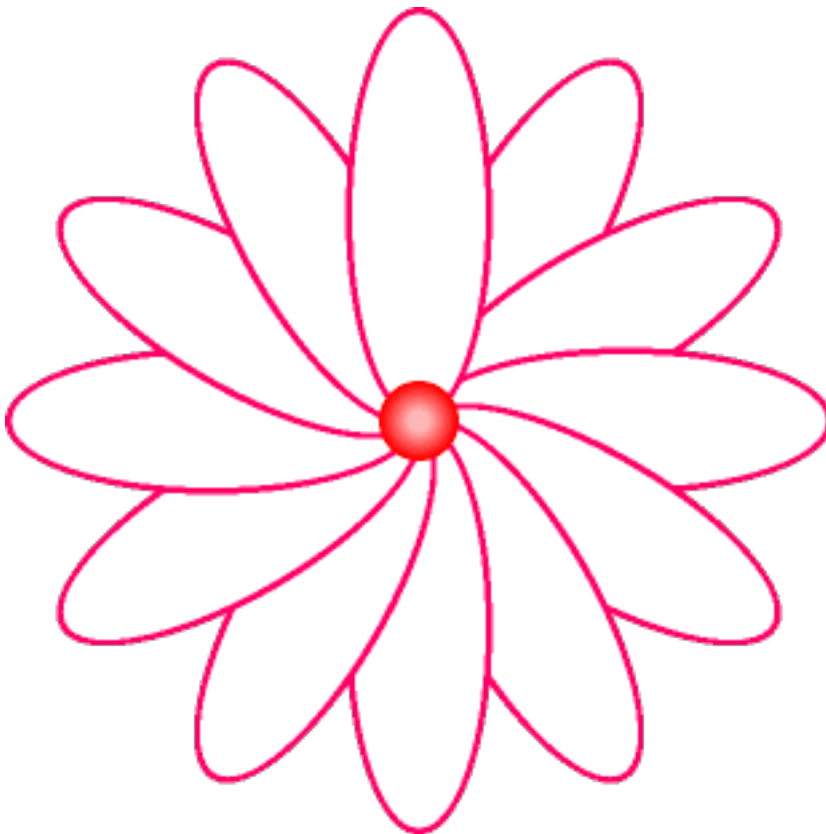
zeige_grafik(scheibe)
```



5.1.6. Schritt 6: Kombination der Elemente zur vollständigen Blume

Zum Schluss wird die Scheibe auf die Blütenblätter gelegt und Blume so vervollständigt.

```
blume_komplett = ueberlagere(scheibe, blume_weiss)  
zeige_grafik(blume_komplett)
```



6. Bedingungen in Python

Es gibt Programmieraufgaben, bei denen einzelne Anweisungen nur ausgeführt werden sollen, wenn bestimmte Bedingungen erfüllt sind. Allgemein funktioniert das nach dem Schema

```
WENN bedingung WAHR
    MACHE
SONST
    MACHE
```

Im folgenden Beispiel wird der Computer mit Python angewiesen, zu prüfen, ob der einer Variablen zugewiesene Wert durch drei Teilbar ist. Wenn dies der Fall ist, soll der Text `Die Zahl ist durch 3 teilbar.` ausgegeben werden, anderenfalls `Die Zahl ist nicht durch 3 teilbar..`

```
zahl = 15

if zahl % 3 == 0:
    print('Die Zahl ist durch 3 teilbar.')
else:
    print('Die Zahl ist nicht durch 3 teilbar..')
```

Die Zahl ist durch 3 teilbar.

Es können auch mehrere Bedingungen nacheinander geprüft werden. Zur Demonstration wird das obige Beispiel um die Prüfung der Teilbarkeit durch fünf ergänzt.

```
zahl = 25

if zahl % 3 == 0:
    print('Die Zahl ist durch 3 teilbar.')
elif zahl % 5 == 0:
    print('Die Zahl ist durch 5 teilbar.')
else:
    print('Die Zahl ist weder durch 3 noch durch 5 teilbar..')
```

Die Zahl ist durch 5 teilbar.

Allerdings werfen die beiden obigen Beispiele zusammen ein neues Problem auf: Was wenn eine Zahl sowohl durch drei als auch durch fünf Teilbar ist?

Es kann auch das Vorliegen mehrerer Bedingungen gleichzeitig geprüft werden:

```
zahl = 15

if zahl % 3 == 0 and zahl % 5 == 0:
    print('Die Zahl ist durch drei und durch fünf teilbar.')
elif zahl % 3 == 0:
    print('Die Zahl ist nur durch drei teilbar.')
elif zahl % 5 == 0:
    print('Die Zahl ist nur durch fünf teilbar.')
else:
    print('Die Zahl ist weder durch drei noch durch fünf teilbar.')
```

Die Zahl ist durch drei und durch fünf teilbar.

Es ist auch möglich, zu prüfen, ob eine Zahl durch drei oder durch fünf teilbar ist:

```
zahl = 27

if zahl % 3 == 0 or zahl % 5 == 0:
    print('Die Zahl ist entweder durch drei oder durch fünf teilbar (oder beides).')
else:
    print('Die Zahl ist weder durch drei noch durch fünf teilbar.')
```

Die Zahl ist entweder durch drei oder durch fünf teilbar (oder beides).

Die wissenschaftliche Disziplin, die sich mit der Kombination des Wahrheitsgehalts von verschiedenen Aussagen beschäftigt, ist die Aussagenlogik.

i Aussagenlogik

Ob die Kombination zweier Aussagen insgesamt wahr oder falsch ist, hängt vom logischen Operator ab, mit dem sie verbunden werden. Gezeigt wird dies oft in sogenannten Wahrheitstabellen. In diesen Wahrheitstabellen werden wahre Aussagen häufig mit einer 1 und falsche Aussagen mit einer 0 gekennzeichnet.

6.1. Der nicht-Operator ($\neg$)

Der nicht-Operator ist ein sogenannter unärer Operator, weil er ausschliesslich einen Operanden braucht. Die Wahrheitstabelle für den nicht-Operator sieht folgendermassen

aus:

$A \quad \neg A \text{ oder } \bar{A}$	
1	0
0	1

6.2. Der und-Operator ($\wedge$)

Die Konjunktion (und-Operator) verbindet zwei Aussagen miteinander. Dabei ist die Aussage der beiden Aussagen im Verbund nur dann wahr, wenn auch die beiden einzelnen Aussagen wahr sind.

A	B	$A \wedge B$
1	1	1
1	0	0
0	1	0
0	0	0

6.3. Der oder-Operator ($\vee$)

Die Disjunktion (oder-Operator) verbindet zwei Aussagen miteinander. Dabei ist die Aussage der beiden Aussagen im Verbund wahr, sobald mindestens eine der Aussagen im Verbund wahr ist.

A	B	$A \vee B$
1	1	1
1	0	1
0	1	1
0	0	0

6.4. Anwendungsbeispiel

Angewandt auf die Blume vom letzten Mal, kann mit Bedingungen eine Blume gezeichnet werden, welche Blütenblätter in unterschiedlichen Farben aufweist.

```
from pytamaro.de import *
```

```
weisses_blatt = ellipse(50, 150, weiss)
rosa_rand = ellipse(55, 155, rgb_farbe(255, 0, 102))

bluetenblatt_mit_rand = ueberlagere(weisses_blatt, rosa_rand)
```

```
standardbluetenblatt = fixiere(unten_mitte, bluetenblatt_mit_rand)

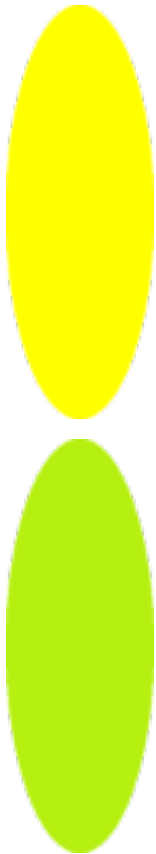
gruenes_blatt = ellipse(55, 155, rgb_farbe(128, 255, 0))
dreier_blatt = fixiere(unten_mitte, gruenes_blatt)

gelbes_blatt = ellipse(55, 155, rgb_farbe(255, 255, 0))
fuenfer_blatt = fixiere(unten_mitte, gelbes_blatt)

kombi_blatt = ellipse(55, 155, rgb_farbe(181, 240, 16))
kombi_blatt = fixiere(unten_mitte, kombi_blatt)

zeige_grafik(standardbluetenblatt)
zeige_grafik(dreier_blatt)
zeige_grafik(fuenfer_blatt)
zeige_grafik(kombi_blatt)
```





```
anzahl_blaetter = 36

blume = kombi_blatt

for i in range(anzahl_blaetter):
    winkel = i * 360 / anzahl_blaetter
    standard_gedreht = drehe(winkel, standardbluetenblatt)
    dreier_gedreht = drehe(winkel, dreier_blatt)
    fuenfer_gedreht = drehe(winkel, fuenfer_blatt)
    kombi_gedreht = drehe(winkel, kombi_blatt)
    if i % 3 == 0 and i % 5 == 0:
        blume = kombiniere(blume, kombi_gedreht)
    elif i % 3 == 0:
        blume = kombiniere(blume, dreier_gedreht)
    elif i % 5 == 0:
        blume = kombiniere(blume, fuenfer_gedreht)
    else:
        blume = kombiniere(blume, standard_gedreht)

zeige_grafik(blume)
```



```
scheibe = ellipse(30, 30, rgb_farbe(255, 0, 0))

for i in range(1, 8):
    tmp = ellipse(30 - 3*i, 30 - 3*i, rgb_farbe(255, i*26, i*26))
    scheibe = ueberlagere(tmp, scheibe)

zeige_grafik(scheibe)
```



```
blume = ueberlagere(scheibe, blume)
zeige_grafik(blume)
```



7. Farbstiftschachtel (verschachtelte for-schleifen)

Schweizer Schulkinder werden mit Prismalo-Farbstiften von Caran D'Ache sozialisiert. Praktisch alle kennen die Schachtel mit sechs Farbstiften.

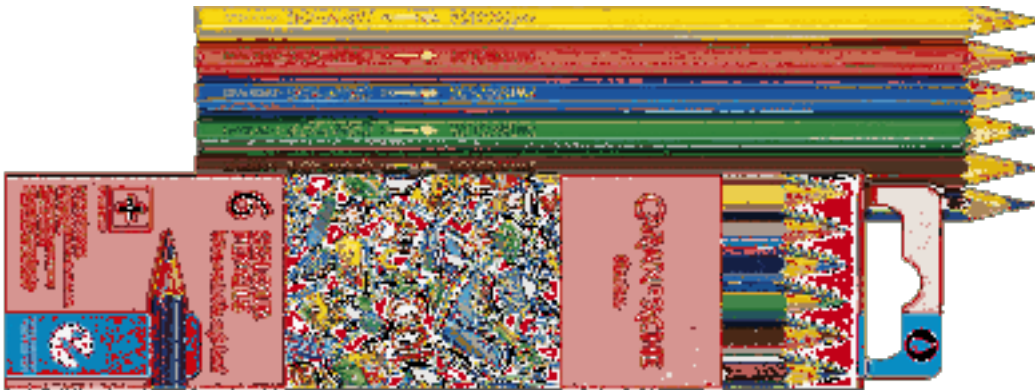


Abbildung 7.1.: Caran D'Ache Farbstifte

In dieser Übung werden wir mit Sechser-Schachtel anfangen und uns bis zur Schachtel mit 120 Farbstiften hocharbeiten. Damit der Aufwand für die Grundstruktur des Farbstiftes nicht zu gross wird, stelle ich Ihnen unten eine Funktion `farbstift()` zur Verfügung. Sie nimmt den Rot-, Grün- und Blauwert der RGB-Farben als Argumente entgegen und gibt einen entsprechend eingefärbten stilisierten Farbstift zurück.

```
from pytamaro.de import *
```

```
def farbstift(r, g, b):
    farbe = rgb_farbe(r, g, b)
    schaft = rechteck(200, 25, farbe)
    holzspitze = dreieck(25, 25, 60, rgb_farbe(238,232,170))
    farbspitze = dreieck(10, 10, 60, farbe)
    holzspitze_fixiert = fixiere(oben_mitte, holzspitze)
    farbspitze_fixiert = fixiere(oben_mitte, farbspitze)
    spitze = kombiniere(farbspitze_fixiert, holzspitze_fixiert)
    spitze_ausgerichtet = drehe(-90, spitze)
    farbstift_gespitzt = neben(schaft, spitze_ausgerichtet)
    return farbstift_gespitzt
```

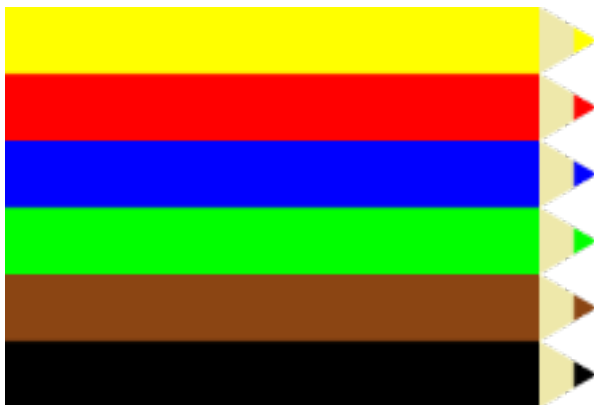
7.1. Kartonschachtel 6 wasservermalbare Farbstifte SCHOOL LINE

Als Beispiel zeichne ich hier die Schachtel der obigen Abbildung.

```
gelber_farbstift    = farbstift(255, 255,  0)
roter_farbstift     = farbstift(255,  0,  0)
blauer_farbstift    = farbstift(  0,  0, 255)
gruener_farbstift   = farbstift(  0, 255,  0)
brauner_farbstift   = farbstift(139, 69, 19)
schwarzer_farbstift = farbstift(  0,  0,  0)

schachtel = leere_grafik()
schachtel1 = ueber(gelber_farbstift, schachtel)
schachtel2 = ueber(schachtel1, roter_farbstift)
schachtel3 = ueber(schachtel2, blauer_farbstift)
schachtel4 = ueber(schachtel3, gruener_farbstift)
schachtel5 = ueber(schachtel4, brauner_farbstift)
schachtel6 = ueber(schachtel5, schwarzer_farbstift)

zeige_grafik(schachtel6)
```



7.2. Etui 120 Farben SUPRACOLOR Aquarelle

Für eine Schachtel mit sechs Farbstiften funktioniert die manuelle Methode leidlich. Die Schachtel mit 120 Farbstiften ist aber sehr aufwändig.

Wie wir gesehen haben, können repetitive Aufgaben mit `for`-Schleifen automatisiert werden. Für die Farbstifte müssen drei Änderungen parallel vorgenommen werden. Dazu können mehrere `for`-Schleifen ineinander verschachtelt werden.

```
# TODO: verschachtelte Schleife für 120 Farbstifte
```